Algoritmos de Odenamiento

Generado por Doxygen 1.15.0

1 Índice jerárquico 1
1.1 Jerarquía de clases . . . 1

2 Índice de clases 3
2.1 Lista de clases . . . 3

3 Índice de archivos 5
3.1 Lista de archivos . . . 5

4 Documentación de clases 7
4.1 Referencia de la clase Client . . . 7
4.1.1 Descripción detallada . . . 7
4.1.2 Documentación de constructores y destructores . . . 8
4.1.2.1 Client() . . . 8
4.1.2.2 ∼Client() . . . 9
4.1.3 Documentación de funciones miembro . . . 9
4.1.3.1 ClientRequestFigure() . . . 9
4.1.3.2 ClientRequestList() . . . 9
4.1.3.3 getId() . . . 10
4.1.3.4 pedirFigura() . . . 10
4.1.4 Documentación de datos miembro . . . 10
4.1.4.1 id . . . 10
4.2 Referencia de la clase Logger . . . 10
4.2.1 Descripción detallada . . . 11
4.2.2 Documentación de constructores y destructores . . . 11
4.2.2.1 Logger() . . . 11
4.2.2.2 ∼Logger() . . . 11
4.2.3 Documentación de funciones miembro . . . 12
4.2.3.1 loadTime() . . . 12
4.2.3.2 log() . . . 12
4.2.3.3 logv() . . . 12
4.2.4 Documentación de datos miembro . . . 12
4.2.4.1 fecha . . . 12
4.2.4.2 logs . . . 12
4.2.4.3 tiempo . . . 13
4.2.4.4 tm . . . 13
4.3 Referencia de la clase Parser . . . 13
4.3.1 Descripción detallada . . . 13
4.3.2 Documentación de constructores y destructores . . . 14
4.3.2.1 Parser() . . . 14
4.3.2.2 ∼Parser() . . . 14
4.3.3 Documentación de funciones miembro . . . 14
4.3.3.1 getFiguras() . . . 14

4.3.3.2 getPiezas() . . . 14
4.3.3.3 procesarFiguras() . . . 14
4.3.3.4 procesarPiezas() . . . 15
4.3.4 Documentación de datos miembro . . . 15
4.3.4.1 Figuras . . . 15
4.3.4.2 piezas . . . 15
4.4 Referencia de la clase Socket . . . 15
4.4.1 Descripción detallada . . . 16
4.4.2 Documentación de constructores y destructores . . . 16
4.4.2.1 Socket() . . . 16
4.4.2.2 ~Socket() . . . 17
4.4.3 Documentación de funciones miembro . . . 17
4.4.3.1 Connect() [1/2] . . . 17
4.4.3.2 Connect() [2/2] . . . 17
4.4.3.3 Read() . . . 18
4.4.3.4 Write() [1/2] . . . 19
4.4.3.5 Write() [2/2] . . . 19
4.5 Referencia de la clase SSLSocket . . . 19
4.5.1 Descripción detallada . . . 21
4.5.2 Documentación de constructores y destructores . . . 21
4.5.2.1 SSLSocket() [1/3] . . . 21
4.5.2.2 SSLSocket() [2/3] . . . 21
4.5.2.3 SSLSocket() [3/3] . . . 21
4.5.2.4 ~SSLSocket() . . . 22
4.5.3 Documentación de funciones miembro . . . 22
4.5.3.1 Connect() [1/2] . . . 22
4.5.3.2 Connect() [2/2] . . . 22
4.5.3.3 GetCipher() . . . 22
4.5.3.4 InitContext() . . . 23
4.5.3.5 InitSSL() . . . 23
4.5.3.6 LoadCertificates() . . . 23
4.5.3.7 Read() . . . 23
4.5.3.8 ShowCerts() . . . 24
4.5.3.9 Write() [1/2] . . . 24
4.5.3.10 Write() [2/2] . . . 24
4.5.4 Documentación de datos miembro . . . 25
4.5.4.1 BIO . . . 25
4.5.4.2 Context . . . 25
4.6 Referencia de la clase VSocket . . . 25
4.6.1 Descripción detallada . . . 26
4.6.2 Documentación de constructores y destructores . . . 26
4.6.2.1 ~VSocket() . . . 26

4.6.3 Documentación de funciones miembro . . . 26
4.6.3.1 Bind() . . . 26
4.6.3.2 Close() . . . 26
4.6.3.3 Connect() [1/2] . . . 26
4.6.3.4 Connect() [2/2] . . . 27
4.6.3.5 Init() . . . 27
4.6.3.6 Read() . . . 27
4.6.3.7 recvFrom() . . . 28
4.6.3.8 sendTo() . . . 28
4.6.3.9 TryToConnect() [1/2] . . . 29
4.6.3.10 TryToConnect() [2/2] . . . 29
4.6.3.11 Write() [1/2] . . . 29
4.6.3.12 Write() [2/2] . . . 29
4.6.4 Documentación de datos miembro . . . 30
4.6.4.1 IPv6 . . . 30
4.6.4.2 port . . . 30
4.6.4.3 sockId . . . 30
4.6.4.4 type . . . 30

5 Documentación de archivos **31**
5.1 Referencia del archivo Client.cpp . . . 31
5.1.1 Descripción detallada . . . 31
5.2 Referencia del archivo Client.hpp . . . 31
5.2.1 Descripción detallada . . . 32
5.2.2 Documentación de «define» . . . 32
5.2.2.1 CLIENT_HPP . . . 32
5.2.2.2 MAXBUF . . . 32
5.3 Client.hpp . . . 32
5.4 Referencia del archivo Logger.cpp . . . 32
5.4.1 Descripción detallada . . . 33
5.5 Referencia del archivo Logger.hpp . . . 33
5.5.1 Descripción detallada . . . 33
5.5.2 Documentación de enumeraciones . . . 33
5.5.2.1 EVENT . . . 33
5.5.2.2 Nodo . . . 33
5.6 Logger.hpp . . . 34
5.7 Referencia del archivo main.cpp . . . 34
5.7.1 Descripción detallada . . . 35
5.7.2 Documentación de «define» . . . 35
5.7.2.1 MAXBUF . . . 35
5.7.3 Documentación de funciones . . . 35
5.7.3.1 main() . . . 35

5.8 Referencia del archivo Parser.cpp 36
5.8.1 Descripción detallada 36
5.9 Referencia del archivo Parser.hpp 36
5.9.1 Descripción detallada 36
5.10 Parser.hpp 36
5.11 Referencia del archivo Socket.cpp 37
5.11.1 Descripción detallada 37
5.12 Referencia del archivo Socket.hpp 37
5.12.1 Descripción detallada 37
5.13 Socket.hpp 37
5.14 Referencia del archivo SSLSocket.cpp 38
5.14.1 Descripción detallada 38
5.15 Referencia del archivo SSLSocket.hpp 38
5.15.1 Descripción detallada 38
5.16 SSLSocket.hpp 38
5.17 Referencia del archivo VSocket.cpp 39
5.17.1 Descripción detallada 39
5.18 Referencia del archivo VSocket.hpp 39
5.18.1 Descripción detallada 39
5.19 VSocket.hpp 40

Índice alfabético **41**

Capítulo 1

Índice jerárquico

1.1. Jerarquía de clases

Este listado de herencia está ordenado de forma general pero no está en orden alfabético estricto:

Client 7
Logger 10
Parser 13
VSocket 25
  SSLSocket 19
  Socket 15

Capítulo 2

Índice de clases

2.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

Client
Clase cliente 7
Logger
Clase que administra la bitacora de eventos del sistema cliente - intermediario - servidor . . . 10
Parser
Clase Parser, procesa html del servidor 13
Socket
Clase Socket 15
SSLSocket
Clase SSLSocket 19
VSocket
Clase virtual VSocket 25

Capítulo 3

Índice de archivos

3.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

Client.cpp
Implementación del client 31
Client.hpp
Definición de la clase Client 31
Logger.cpp
Implementación de clase Logger 32
Logger.hpp
Definición de la clase Logger 33
main.cpp
Creación del cliente y interfaz para interactuar 34
Parser.cpp
Implementación de clase Parser 36
Parser.hpp
Definición de clase parser 36
Socket.cpp
Implementación de un socket normal 37
Socket.hpp
Definición de un socket normal 37
SSLSocket.cpp
Implementación de la clase SSLSocket 38
SSLSocket.hpp
Definición de la clase SSLSocket 38
VSocket.cpp
Implementacion de la clase virtual VSocket 39
VSocket.hpp
Definición de la clase virtual VSocket 39

Capítulo 4

Documentación de clases

4.1. Referencia de la clase Client

Clase cliente.

```
#include <Client.hpp>
```

Métodos públicos

- Client (int id)

  Constructor del Client.
- ~Client ()

  Destructor del Client.
- void ClientRequestFigure (VSocket *client, std::string figuraElegida, int parteElegida, const char *service, Logger &log)

  Solicita una figura.
- void ClientRequestList (VSocket *client, const char *service, Logger &log)

  Solicita la lista de figuras.
- std::vector< std::pair< std::string, int > > pedirFigura (std::string figura, int parte, int argc, const char *service, Parser &parser, Logger &log)

  Solicita la figura y la procesa.
- int getId ()

  Devuelve la id del cliente.

Atributos privados

- int id

4.1.1. Descripción detallada

Clase cliente.

Representa el cliente que realiza solicitudes al servidor de piezas

4.1.2. Documentación de constructores y destructores

4.1.2.1. Client()

```
Client::Client (
            int id)
```

Constructor del Client.

Parámetros

Id	del cliente

4.1.2.2. ~Client()

```
Client::~Client ()
```

Destructor del Client.

4.1.3. Documentación de funciones miembro

4.1.3.1. ClientRequestFigure()

```
void Client::ClientRequestFigure (
            VSocket * client,
            std::string figuraElegida,
            int parteElegida,
            const char * service,
            Logger & log)
```

Solicita una figura.

Parámetros

client	El cliente que solicita
figuraElegida	Figura a solicitar
parteElegida	Parte de la figura a solicitar
service	Servicio https o http
log	Referencia al Logger

4.1.3.2. ClientRequestList()

```
void Client::ClientRequestList (
            VSocket * client,
            const char * service,
            Logger & log)
```

Solicita la lista de figuras.

Parámetros

client	El cliente que solicita
service	Servicio https o http
log	Referencia al Logger

4.1.3.3. getId()

```
int Client::getId ()
```

Devuelve la id del cliente.

4.1.3.4. pedirFigura()

```
std::vector< std::pair< std::string, int > > Client::pedirFigura (
            std::string figura,
            int parte,
            int argc,
            const char * service,
            Parser & parser,
            Logger & log)
```

Solicita la figura y la procesa.

Parámetros

figura	Figura a pedir
parte	Parte de la figura
argc	Contador de argumentos > 1 ? https: http
service	Servicio https o http
parser	Referencia al parser para procesar html
log	Referencia al logger

Devuelve

Vector de pares de descripcion de pieza y cantidad

4.1.4. Documentación de datos miembro

4.1.4.1. id

```
int Client::id [private]
```

La documentación de esta clase está generada de los siguientes archivos:

- Client.hpp
- Client.cpp

4.2. Referencia de la clase Logger

Clase que administra la bitacora de eventos del sistema cliente - intermediario - servidor.

```
#include <Logger.hpp>
```

Métodos públicos

- Logger (std::string dir)

  Constructor de la clase Logger.
- ~Logger ()

  Destructor por defecto de la clase Logger.
- void log (std::string txt, Nodo t=Cliente)

  Metodo que registra un evento en la bitacora.
- void logv (std::vector< std::string > &txt, Nodo t)

  Metodo que registra un evento (en formato vector<string>) en la bitacora.

Métodos privados

- void loadTime ()

  Metodo que actualiza la fecha y hora de un evento,.

Atributos privados

- std::ofstream logs
- std::time_t tiempo
- std::tm * tm
- char fecha [80]

4.2.1. Descripción detallada

Clase que administra la bitacora de eventos del sistema cliente - intermediario - servidor.

4.2.2. Documentación de constructores y destructores

4.2.2.1. Logger()

```
Logger::Logger (
            std::string dir)
```

Constructor de la clase Logger.

Parámetros

dir	Directorio del archivo donde se registrara la bitacora.

4.2.2.2. ~Logger()

```
Logger::~Logger ()
```

Destructor por defecto de la clase Logger.

4.2.3. Documentación de funciones miembro

4.2.3.1. loadTime()

```
void Logger::loadTime () [private]
```

Metodo que actualiza la fecha y hora de un evento,.

4.2.3.2. log()

```
void Logger::log (
            std::string txt,
            Nodo t = Cliente)
```

Metodo que registra un evento en la bitacora.

Parámetros

txt	String con el mensaje del evento.
t	Enum con el nodo que genera el registro.

4.2.3.3. logv()

```
void Logger::logv (
            std::vector< std::string > & txt,
            Nodo t = Cliente)
```

Metodo que registra un evento (en formato vector<string>) en la bitacora.

Parámetros

txt	Vector de strings con el mensaje del evento.
t	Enum con el nodo que genera el registro.

4.2.4. Documentación de datos miembro

4.2.4.1. fecha

```
char Logger::fecha[80] [private]
```

4.2.4.2. logs

```
std::ofstream Logger::logs [private]
```

4.2.4.3. tiempo

```
std::time_t Logger::tiempo [private]
```

4.2.4.4. tm

```
std::tm* Logger::tm [private]
```

La documentación de esta clase está generada de los siguientes archivos:

- Logger.hpp
- Logger.cpp

4.3. Referencia de la clase Parser

Clase Parser, procesa html del servidor.

```
#include <Parser.hpp>
```

Métodos públicos

- Parser ()

  Constructor del parser.
- ~Parser ()

  Destructor del parser.
- void procesarFiguras (const std::string &html)

  Extrae la lista de figuras.
- void procesarPiezas (const std::string &html)

  Extrae las piezas.
- std::vector< std::string > getFiguras () const

  Devuelve las figuras.
- std::vector< std::pair< std::string, int > > getPiezas () const

  Devuelve las figuras.

Atributos privados

- std::vector< std::string > Figuras
- std::vector< std::pair< std::string, int > > piezas

4.3.1. Descripción detallada

Clase Parser, procesa html del servidor.

Encargado de procesar el html que devuelve el servidor para extraer la lista de figuras y las piezas del html

4.3.2. Documentación de constructores y destructores

4.3.2.1. Parser()

```
Parser::Parser ()
```

Constructor del parser.

4.3.2.2. ~Parser()

```
Parser::~Parser ()
```

Destructor del parser.

4.3.3. Documentación de funciones miembro

4.3.3.1. getFiguras()

```
std::vector< std::string > Parser::getFiguras () const
```

Devuelve las figuras.

Devuelve

Vector de figuras

4.3.3.2. getPiezas()

```
std::vector< std::pair< std::string, int > > Parser::getPiezas () const
```

Devuelve las figuras.

Devuelve

Par de pieza y cantidad

4.3.3.3. procesarFiguras()

```
void Parser::procesarFiguras (
            const std::string & html)
```

Extrae la lista de figuras.

Parámetros

html	Documento html en formato string

4.3.3.4. procesarPiezas()

```
void Parser::procesarPiezas (
            const std::string & html)
```

Extrae las piezas.

Parámetros

html	String html de la pieza

4.3.4. Documentación de datos miembro

4.3.4.1. Figuras

```
std::vector<std::string> Parser::Figuras [private]
```

4.3.4.2. piezas

```
std::vector<std::pair<std::string, int> > Parser::piezas [private]
```

Lista de figuras

La documentación de esta clase está generada de los siguientes archivos:

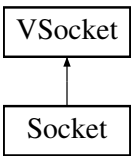
- Parser.hpp
- Parser.cpp

4.4. Referencia de la clase Socket

Clase Socket.

```
#include <Socket.hpp>
```

Diagrama de herencia de Socket



Métodos públicos

- Socket (char, bool=false)
- ~Socket ()
- int Connect (const char *, int)
- int Connect (const char *, const char *)
- size_t Read (void *, size_t)
- size_t Write (const void *, size_t)
- size_t Write (const char *)

Métodos públicos heredados de VSocket

- void Init (char type, bool IPv6=false)
- virtual ~VSocket ()

  Destructo de VSocket.
- void Close ()
- int TryToConnect (const char *hostip, int port)
- int TryToConnect (const char *hostip, const char *service)
- int Bind (int port)
- size_t sendTo (const void *, size_t, void *)
- size_t recvFrom (void *, size_t, void *)

Otros miembros heredados

Atributos protegidos heredados de VSocket

- int sockId
- bool IPv6

  Socket identifier.
- int port

  Is IPv6 socket?
- char type

  Socket associated port.

4.4.1. Descripción detallada

Clase Socket.

Se encarga de construir un socket de conexion sin ssl para conexiones tipo udp o tcp

4.4.2. Documentación de constructores y destructores

4.4.2.1. Socket()

```
Socket::Socket (
            char t,
            bool IPv6 = false)
```

Class constructor use Unix socket system call

Parámetros

char	t: socket type to define 's' for stream 'd' for datagram
bool	ipv6: if we need a IPv6 socket

4.4.2.2. ~Socket()

```
Socket::~Socket ()
```

Class destructor

Parámetros

int	id: socket descriptor

4.4.3. Documentación de funciones miembro

4.4.3.1. Connect() [1/2]

```
int Socket::Connect (
            const char * host,
            const char * service)  [virtual]
```

Connect method use "TryToConnect" in base class

Parámetros

char	* host: host address in dns notation, example "os.ecci.ucr.ac.cr"
char	* service: process address, example "http"

Implementa VSocket.

4.4.3.2. Connect() [2/2]

```
int Socket::Connect (
            const char * hostip,
            int port)  [virtual]
```

Connect method use "TryToConnect" in base class

Parámetros

char	* host: host address in dot notation, example "10.1.166.62"
int	port: process address, example 80

Implementa VSocket.

4.4.3.3. Read()

```
size_t Socket::Read (
            void * buffer,
            size_t size)  [virtual]
```

Read method use "read" Unix system call (man 3 read)

Parámetros

---

void	∗ buffer: buffer to store data read from socket
int	size: buffer capacity, read will stop if buffer is full

Implementa VSocket.

4.4.3.4. Write() [1/2]

```
size_t Socket::Write (
            const char * text)  [virtual]
```

Write method use "write" Unix system call (man 3 write)

Parámetros

char	∗ text: text to write to socket

Implementa VSocket.

4.4.3.5. Write() [2/2]

```
size_t Socket::Write (
            const void * buffer,
            size_t size)  [virtual]
```

Write method use "write" Unix system call (man 3 write)

Parámetros

void	∗ buffer: buffer to store data write to socket
size↩ _t	size: buffer capacity, number of bytes to write

Implementa VSocket.

La documentación de esta clase está generada de los siguientes archivos:

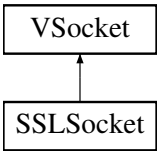
- Socket.hpp
- Socket.cpp

4.5. Referencia de la clase SSLSocket

Clase SSLSocket.

```
#include <SSLSocket.hpp>
```

Diagrama de herencia de SSLSocket



Métodos públicos

- SSLSocket (bool IPv6=false)
- SSLSocket (char ∗, char ∗, bool=false)
- SSLSocket (int)
- ∼SSLSocket ()
- int Connect (const char ∗, int)
- int Connect (const char ∗, const char ∗)
- size_t Write (const char ∗)
- size_t Write (const void ∗, size_t)
- size_t Read (void ∗, size_t)
- void ShowCerts ()
- const char ∗ GetCipher ()

Métodos públicos heredados de VSocket

- void Init (char type, bool IPv6=false)
- virtual ∼VSocket ()

  Destructo de VSocket.
- void Close ()
- int TryToConnect (const char ∗hostip, int port)
- int TryToConnect (const char ∗hostip, const char ∗service)
- int Bind (int port)
- size_t sendTo (const void ∗, size_t, void ∗)
- size_t recvFrom (void ∗, size_t, void ∗)

Métodos privados

- void InitSSL (bool=false)
- void InitContext (bool)
- void LoadCertificates (const char ∗, const char ∗)

Atributos privados

- void ∗ Context
- void ∗ BIO

  SSL context.

Otros miembros heredados

Atributos protegidos heredados de VSocket

- int sockId
- bool IPv6

  Socket identifier.
- int port

  Is IPv6 socket?
- char type

  Socket associated port.

4.5.1. Descripción detallada

Clase SSLSocket.

Construye un socket usando certificados ssl para comunicación segura entre dispositivos

4.5.2. Documentación de constructores y destructores

4.5.2.1. SSLSocket() [1/3]

```
SSLSocket::SSLSocket (
            bool IPv6 = false)
```

Class constructor use base class

Parámetros

char	t: socket type to define 's' for stream 'd' for datagram
bool	ipv6: if we need a IPv6 socket

4.5.2.2. SSLSocket() [2/3]

```
SSLSocket::SSLSocket (
            char * certFileName,
            char * keyFileName,
            bool IPv6 = false)
```

Class constructor use base class

Parámetros

char	t: socket type to define 's' for stream 'd' for datagram
bool	IPv6: if we need a IPv6 socket

4.5.2.3. SSLSocket() [3/3]

```
SSLSocket::SSLSocket (
            int id)
```

Class constructor

Parámetros

int	id: socket descriptor

4.5.2.4. ~SSLSocket()

```
SSLSocket::~SSLSocket ()
```

Class destructor

4.5.3. Documentación de funciones miembro

4.5.3.1. Connect() [1/2]

```
int SSLSocket::Connect (
            const char * host,
            const char * service)  [virtual]
```

Connect use SSL_connect to establish a secure conection

Create a SSL connection

Parámetros

char	* hostName, host name
char	* service, service name

Implementa VSocket.

4.5.3.2. Connect() [2/2]

```
int SSLSocket::Connect (
            const char * hostName,
            int port)  [virtual]
```

Connect use SSL_connect to establish a secure conection

Create a SSL connection

Parámetros

char	* hostName, host name
int	port, service number

Implementa VSocket.

4.5.3.3. GetCipher()

```
const char * SSLSocket::GetCipher ()
```

Return the name of the currently used cipher

4.5.3.4. InitContext()

```
void SSLSocket::InitContext (
            bool serverContext)  [private]
```

InitContext use SSL_library_init, OpenSSL_add_all_algorithms, SSL_load_error_strings, TLS_server_method, SSL_CTX_new

Creates a new SSL server context to start encrypted comunications, this context is stored in class instance

4.5.3.5. InitSSL()

```
void SSLSocket::InitSSL (
            bool serverContext = false)  [private]
```

InitSSL use SSL_new with a defined context

Create a SSL object

4.5.3.6. LoadCertificates()

```
void SSLSocket::LoadCertificates (
            const char * certFileName,
            const char * keyFileName)  [private]
```

Load certificates verify and load certificates

Parámetros

const	char ∗ certFileName, file containing certificate
const	char ∗ keyFileName, file containing keys

4.5.3.7. Read()

```
size_t SSLSocket::Read (
            void * buffer,
            size_t size)  [virtual]
```

Read use SSL_read to read data from an encrypted channel

Parámetros

void	∗ buffer to store data read
size↩ _t	size, buffer's capacity

Devuelve

size_t byte quantity read

Reads data from secure channel

Implementa VSocket.

4.5.3.8. ShowCerts()

```
void SSLSocket::ShowCerts ()
```

Show SSL certificates

4.5.3.9. Write() [1/2]

```
size_t SSLSocket::Write (
            const char * string)  [virtual]
```

Write use SSL_write to write data to an encrypted channel

Parámetros

void	∗ buffer to store data read
size↩ _t	size, buffer's capacity

Devuelve

size_t byte quantity written

Writes data to a secure channel

Implementa VSocket.

4.5.3.10. Write() [2/2]

```
size_t SSLSocket::Write (
            const void * buffer,
            size_t size)  [virtual]
```

Write use SSL_write to write data to an encrypted channel

Parámetros

const	void ∗ buffer to store data to write
size↩ _t	size, buffer's capacity

Devuelve

size_t byte quantity written

Reads data from secure channel

Implementa VSocket.

4.5.4. Documentación de datos miembro

4.5.4.1. BIO

```
void* SSLSocket::BIO [private]
```

SSL context.

4.5.4.2. Context

```
void* SSLSocket::Context [private]
```

La documentación de esta clase está generada de los siguientes archivos:

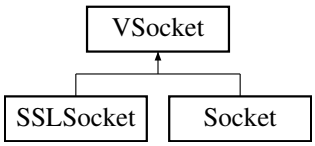
- SSLSocket.hpp
- SSLSocket.cpp

4.6. Referencia de la clase VSocket

Clase virtual VSocket.

```
#include <VSocket.hpp>
```

Diagrama de herencia de VSocket



Métodos públicos

- void Init (char type, bool IPv6=false)
- virtual ~VSocket ()

  Destructo de VSocket.
- void Close ()
- int TryToConnect (const char *hostip, int port)
- int TryToConnect (const char *hostip, const char *service)
- virtual int Connect (const char *, int)=0

  Metodo virtual de conexion por puerto.
- virtual int Connect (const char *, const char *)=0

  Metodo virtual de conexion por servicio.
- virtual size_t Read (void *buffer, size_t size)=0

  Metodo virtual de lectura de mensajes.
- virtual size_t Write (const void *buffer, size_t size)=0

  Metodo virtual de escritura de mensajes.
- virtual size_t Write (const char *buffer)=0

  Metodo virtual de escritura de mensajes.
- int Bind (int port)
- size_t sendTo (const void *, size_t, void *)
- size_t recvFrom (void *, size_t, void *)

Atributos protegidos

- int sockId
- bool IPv6

  Socket identifier.
- int port

  Is IPv6 socket?
- char type

  Socket associated port.

4.6.1. Descripción detallada

Clase virtual VSocket.

Se encarga de manejar los metodos de conexión del socket, así como los metodos de envió y recepción de mensajes udp

4.6.2. Documentación de constructores y destructores

4.6.2.1. ~VSocket()

```
VSocket::~VSocket () [virtual]
```

Destructo de VSocket.

Class destructor

4.6.3. Documentación de funciones miembro

4.6.3.1. Bind()

```
int VSocket::Bind (
            int port)
```

Bind method use "bind" Unix system call (man 3 bind) (server mode)

Parámetros

int	port: bind a unamed socket to a port defined in sockaddr structure

Links the calling process to a service at port

4.6.3.2. Close()

```
void VSocket::Close ()
```

Close method use Unix close system call (once opened a socket is managed like a file in Unix)

4.6.3.3. Connect() [1/2]

```
virtual int VSocket::Connect (
            const char * ,
            const char * ) [pure virtual]
```

Metodo virtual de conexion por servicio.

Parámetros

hostip	Ip a la que se va a conectar
service	Servicio al que se va a conectar

Devuelve

Devuelve $<$ 0 si falla, 1 si funciona

Implementado en Socket y SSLSocket.

4.6.3.4. Connect() [2/2]

```
virtual int VSocket::Connect (
            const char * ,
            int )  [pure virtual]
```

Metodo virtual de conexion por puerto.

Parámetros

hostip	Ip a la que se va a conectar
port	Puerto al que se va a conectar

Devuelve

Devuelve $<$ 0 si falla, 1 si funciona

Implementado en Socket y SSLSocket.

4.6.3.5. Init()

```
void VSocket::Init (
            char t,
            bool IPv6 = false)
```

Class creator (constructor) use Unix socket system call

Parámetros

char	t: socket type to define 's' for stream 'd' for datagram
bool	ipv6: if we need a IPv6 socket

4.6.3.6. Read()

```
virtual size_t VSocket::Read (
            void * buffer,
            size_t size)  [pure virtual]
```

Metodo virtual de lectura de mensajes.

Parámetros

buffer	Buffer donde se lee el mensaje
size	Tamaño del buffer

Devuelve

Cantidad bytes leidos

Implementado en Socket y SSLSocket.

4.6.3.7. recvFrom()

```
size_t VSocket::recvFrom (
            void * buffer,
            size_t size,
            void * addr)
```

recvFrom method

Parámetros

const	void ∗ buffer: data to send
size↩ _t	size data size to send
void	∗ addr address to receive from data

Devuelve

size_t bytes received

Receive data from another network point (addr) without connection (Datagram)

4.6.3.8. sendTo()

```
size_t VSocket::sendTo (
            const void * buffer,
            size_t size,
            void * addr)
```

sendTo method

Parámetros

const	void ∗ buffer: data to send
size↩ _t	size data size to send
void	∗ addr address to send data

Send data to another network point (addr) without connection (Datagram)

4.6.3.9. TryToConnect() [1/2]

```
int VSocket::TryToConnect (
            const char * host,
            const char * service)
```

TryToConnect method use "connect" Unix system call

Parámetros

char	* host: host address in dns notation, example "os.ecci.ucr.ac.cr"
char	* service: process address, example "http"

4.6.3.10. TryToConnect() [2/2]

```
int VSocket::TryToConnect (
            const char * hostip,
            int port)
```

TryToConnect method use "connect" Unix system call

Parámetros

char	* host: host address in dot notation, example "10.84.166.62"
int	port: process address, example 80

4.6.3.11. Write() [1/2]

```
virtual size_t VSocket::Write (
            const char * buffer)  [pure virtual]
```

Metodo virtual de escritura de mensajes.

Parámetros

buffer	Buffer donde se escribe el mensaje

Devuelve

Cantidad bytes escritos

Implementado en Socket y SSLSocket.

4.6.3.12. Write() [2/2]

```
virtual size_t VSocket::Write (
            const void * buffer,
            size_t size)  [pure virtual]
```

Metodo virtual de escritura de mensajes.

Parámetros

buffer	Buffer donde se escribe el mensaje
size	Tamaño del buffer

Devuelve

Cantidad bytes escritos

Implementado en Socket y SSLSocket.

4.6.4. Documentación de datos miembro

4.6.4.1. IPv6

```
bool VSocket::IPv6 [protected]
```

Socket identifier.

4.6.4.2. port

```
int VSocket::port [protected]
```

Is IPv6 socket?

4.6.4.3. sockId

```
int VSocket::sockId [protected]
```

4.6.4.4. type

```
char VSocket::type [protected]
```

Socket associated port.

La documentación de esta clase está generada de los siguientes archivos:

- VSocket.hpp
- VSocket.cpp

Capítulo 5

Documentación de archivos

5.1. Referencia del archivo Client.cpp

Implementación del client.

```
#include <cstring>
#include "Client.hpp"
```

5.1.1. Descripción detallada

Implementación del client.

5.2. Referencia del archivo Client.hpp

Definición de la clase Client.

```
#include <vector>
#include <string>
#include "Parser.hpp"
#include "Logger.hpp"
#include "VSocket.hpp"
#include "SSLSocket.hpp"
#include "Socket.hpp"
```

Clases

- class Client

  Clase cliente.

defines

- #define CLIENT_HPP
- #define MAXBUF 1024

5.2.1. Descripción detallada

Definición de la clase Client.

5.2.2. Documentación de «define»

5.2.2.1. CLIENT_HPP

```
#define CLIENT_HPP
```

5.2.2.2. MAXBUF

```
#define MAXBUF 1024
```

5.3. Client.hpp

Ir a la documentación de este archivo.

```
00001
00005 #ifndef CLIENTE_HPP
00006 #define CLIENT_HPP
00007 #include <vector>
00008 #include <string>
00009 #include "Parser.hpp"
00010 #include "Logger.hpp"
00011 #include "VSocket.hpp"
00012 #include "SSLSocket.hpp"
00013 #include "Socket.hpp"
00014 #define MAXBUF 1024
00015
00021 class Client {
00022     public:
00028         Client(int id);
00029
00033         ~Client();
00034
00044         void ClientRequestFigure(VSocket* client, std::string figuraElegida, int parteElegida, const
      char* service, Logger& log);
00045
00053         void ClientRequestList(VSocket* client, const char* service, Logger& log);
00054
00067         std::vector<std::pair<std::string, int» pedirFigura(std::string figura,
00068         int parte, int argc, const char* service,Parser& parser, Logger& log);
00069
00073         int getId();
00074
00075     private:
00076         int id;
00077 };
00078 #endif //CLIENT_HPP
```

5.4. Referencia del archivo Logger.cpp

Implementación de clase Logger.

```
#include <iostream>
#include <fstream>
#include "Logger.hpp"
#include <ctime>
#include <string>
```

5.4.1. Descripción detallada

Implementación de clase Logger.

5.5. Referencia del archivo Logger.hpp

Definición de la clase Logger.

```
#include <fstream>
#include <string>
#include <ctime>
#include <vector>
```

Clases

- class Logger

  Clase que administra la bitacora de eventos del sistema cliente - intermediario - servidor.

Enumeraciones

- enum Nodo { Cliente , Tenedor , Server , Usuario }

  Tipo de nodo.
- enum EVENT { WARN , ERROR }

5.5.1. Descripción detallada

Definición de la clase Logger.

5.5.2. Documentación de enumeraciones

5.5.2.1. EVENT

```
enum EVENT
```

Valores de enumeraciones

WARN	
ERROR	

5.5.2.2. Nodo

```
enum Nodo
```

Tipo de nodo.

Valores de enumeraciones

Cliente	
Tenedor	
Server	
Usuario	

5.6. Logger.hpp

Ir a la documentación de este archivo.

```
00001
00005 #ifndef LOGGER_HPP
00006 #define LOGGER_HPP
00007 #include <fstream>
00008 #include <string>
00009 #include <ctime>
00010 #include <vector>
00011
00015 enum Nodo {
00016 Cliente,
00017 Tenedor,
00018 Server,
00019 Usuario
00020 };
00021
00022
00026 enum EVENT {
00027   WARN,
00028   ERROR
00029 };
00030
00031
00032
00033
00037 class Logger
00038 {
00039 private:
00040   std::ofstream logs; // Archivo donde se registra la bitacora.
00041   std::time_t tiempo; // Registro de tiempo para la bitacora.
00042   std::tm* tm; // Auxiliar del registro de tiempo.
00043   char fecha[80]; // Buffer donde se guarda la fecha en formato string.
00044
00045 public:
00050   Logger(std::string dir);
00051
00055   ~Logger();
00056
00062   void log(std::string txt, Nodo t = Cliente);
00063
00069   void logv(std::vector<std::string>& txt, Nodo t);
00070 private:
00074   void loadTime();
00075 };
00076
00077
00078
00079
00080 #endif
```

5.7. Referencia del archivo main.cpp

Creación del cliente y interfaz para interactuar.

```
#include <stdio.h>
#include <string.h>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
```

```
#include "VSocket.hpp"
#include "Socket.hpp"
#include "SSLSocket.hpp"
#include "Parser.hpp"
#include "Client.hpp"
#include "Logger.hpp"
```

defines

- #define MAXBUF 1024 /** Tamaño maximo del buffer */

Funciones

- int main (int argc, char *argv[ ])

  Creación del cliente y interfaz para interactuar.

5.7.1. Descripción detallada

Creación del cliente y interfaz para interactuar.

5.7.2. Documentación de «define»

5.7.2.1. MAXBUF

```
#define MAXBUF 1024 /** Tamaño maximo del buffer */
```

5.7.3. Documentación de funciones

5.7.3.1. main()

```
int main (
            int argc,
            char * argv[ ])
```

Creación del cliente y interfaz para interactuar.

Parámetros

argc	Contador de argumentos
argv	Argumentos, si es más de 1 es https de otra manera http

Devuelve

Ejecución exitosa en 0

5.8. Referencia del archivo Parser.cpp

Implementación de clase Parser.

```
#include "Parser.hpp"
#include <regex>
#include <iostream>
```

5.8.1. Descripción detallada

Implementación de clase Parser.

5.9. Referencia del archivo Parser.hpp

Definición de clase parser.

```
#include <vector>
#include <string>
```

Clases

- class Parser

  Clase Parser, procesa html del servidor.

5.9.1. Descripción detallada

Definición de clase parser.

5.10. Parser.hpp

Ir a la documentación de este archivo.

```
00001
00005 #ifndef PARSER_HPP
00006 #define PARSER_HPP
00007
00008 #include <vector>
00009 #include <string>
00017 class Parser {
00018 public:
00019
00023     Parser();
00024
00028     ~Parser();
00029
00035     void procesarFiguras(const std::string &html);
00036
00042     void procesarPiezas(const std::string &html);
00043
00049     std::vector<std::string> getFiguras() const;
00055     std::vector<std::pair<std::string, int» getPiezas() const;
00056
00057
00058 private:
00059     std::vector<std::string> Figuras;
00060     std::vector<std::pair<std::string, int» piezas;
00061 };
00062
00063 #endif //PARSER_HPP
```

5.11. Referencia del archivo Socket.cpp

Implementación de un socket normal.

```
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <cstring>
#include <stdexcept>
#include <stdio.h>
#include "Socket.hpp"
```

5.11.1. Descripción detallada

Implementación de un socket normal.

5.12. Referencia del archivo Socket.hpp

Definición de un socket normal.

```
#include "VSocket.hpp"
#include <cstddef>
```

Clases

- class Socket

  Clase Socket.

5.12.1. Descripción detallada

Definición de un socket normal.

5.13. Socket.hpp

Ir a la documentación de este archivo.

```
00001
00005 #ifndef Socket_hpp
00006 #define Socket_hpp
00007 #include "VSocket.hpp"
00008 #include <cstddef>
00009
00016 class Socket : public VSocket {
00017
00018    public:
00019       Socket( char, bool = false );
00020       ~Socket();
00021       int Connect( const char *, int );
00022       int Connect( const char *, const char * );
00023       size_t Read( void *, size_t );
00024       size_t Write( const void *, size_t );
00025       size_t Write( const char * );
00026
00027    protected:
00028
00029 };
00030
00031 #endif
00032
```

5.14. Referencia del archivo SSLSocket.cpp

Implementación de la clase SSLSocket.

```
#include <openssl/ssl.h>
#include <openssl/err.h>
#include <stdexcept>
#include "SSLSocket.hpp"
#include "Socket.hpp"
```

5.14.1. Descripción detallada

Implementación de la clase SSLSocket.

5.15. Referencia del archivo SSLSocket.hpp

Definición de la clase SSLSocket.

```
#include "VSocket.hpp"
#include <cstddef>
```

Clases

- class SSLSocket

  Clase SSLSocket.

5.15.1. Descripción detallada

Definición de la clase SSLSocket.

5.16. SSLSocket.hpp

Ir a la documentación de este archivo.

```
00001
00005 #ifndef SSLSocket_hpp
00006 #define SSLSocket_hpp
00007
00008 #include "VSocket.hpp"
00009 #include <cstddef>
00010
00017 class SSLSocket : public VSocket {
00018
00019    public:
00020       SSLSocket( bool IPv6 = false );               // Not possible to create with UDP, client
      constructor
00021       SSLSocket( char *, char *, bool = false );        // For server connections
00022       SSLSocket( int );
00023       ~SSLSocket();
00024       int Connect( const char *, int );
00025       int Connect( const char *, const char * );
00026       size_t Write( const char * );
```

```
00027       size_t Write( const void *, size_t );
00028       size_t Read( void *, size_t );
00029       void ShowCerts();
00030       const char * GetCipher();
00031
00032    private:
00033       void InitSSL( bool = false );// Defaults to create a client context, true if server context
      needed
00034       void InitContext( bool );
00035       void LoadCertificates( const char *, const char * );
00036
00037 // Instance variables
00038       void * Context;
00039       void * BIO;
00040
00041 };
00042
00043 #endif
00044
```

5.17. Referencia del archivo VSocket.cpp

Implementacion de la clase virtual VSocket.

```
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdexcept>
#include <cstring>
#include <netdb.h>
#include <unistd.h>
#include "VSocket.hpp"
#include <sys/un.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
```

5.17.1. Descripción detallada

Implementacion de la clase virtual VSocket.

5.18. Referencia del archivo VSocket.hpp

Definición de la clase virtual VSocket.

Clases

- class VSocket

  Clase virtual VSocket.

5.18.1. Descripción detallada

Definición de la clase virtual VSocket.

5.19. VSocket.hpp

Ir a la documentación de este archivo.

```
00001
00005 #ifndef VSocket_hpp
00006 #define VSocket_hpp
00007
00014 class VSocket {
00015    public:
00016        void Init( char type, bool IPv6 = false );
00017
00021        virtual ~VSocket();
00022
00023       void Close();
00024
00025       int TryToConnect( const char * hostip, int port );
00026
00027       int TryToConnect( const char * hostip, const char * service);
00028
00037       virtual int Connect( const char *, int ) = 0;
00038
00047       virtual int Connect( const char *, const char * ) = 0;
00048
00057       virtual size_t Read( void * buffer, size_t size ) = 0;
00058
00067       virtual size_t Write( const void * buffer, size_t size ) = 0;
00068
00076       virtual size_t Write( const char * buffer) = 0;
00077
00078       int Bind( int port );
00079
00080 // UDP methods
00081       size_t sendTo( const void *, size_t, void * );
00082       size_t recvFrom( void *, size_t, void * );
00083
00084    protected:
00085       int sockId;
00086       bool IPv6;
00087       int port;
00088       char type;
00089
00090 };
00091
00092 #endif // VSocket_h
```

Índice alfabético

~Client
  Client, 9
~Logger
  Logger, 11
~Parser
  Parser, 14
~SSLSocket
  SSLSocket, 21
~Socket
  Socket, 17
~VSocket
  VSocket, 26

Bind
  VSocket, 26
BIO
  SSLSocket, 25

Client, 7
  ~Client, 9
  Client, 8
  ClientRequestFigure, 9
  ClientRequestList, 9
  getId, 9
  id, 10
  pedirFigura, 10
Client.cpp, 31
Client.hpp, 31
  CLIENT_HPP, 32
  MAXBUF, 32
CLIENT_HPP
  Client.hpp, 32
Cliente
  Logger.hpp, 34
ClientRequestFigure
  Client, 9
ClientRequestList
  Client, 9
Close
  VSocket, 26
Connect
  Socket, 17
  SSLSocket, 22
  VSocket, 26, 27
Context
  SSLSocket, 25

ERROR
  Logger.hpp, 33
EVENT
  Logger.hpp, 33

fecha
  Logger, 12
Figuras
  Parser, 15

GetCipher
  SSLSocket, 22
getFiguras
  Parser, 14
getId
  Client, 9
getPiezas
  Parser, 14

id
  Client, 10
Init
  VSocket, 27
InitContext
  SSLSocket, 22
InitSSL
  SSLSocket, 23
IPv6
  VSocket, 30

LoadCertificates
  SSLSocket, 23
loadTime
  Logger, 12
log
  Logger, 12
Logger, 10
  ~Logger, 11
  fecha, 12
  loadTime, 12
  log, 12
  Logger, 11
  logs, 12
  logv, 12
  tiempo, 12
  tm, 13
Logger.cpp, 32
Logger.hpp, 33
  Cliente, 34
  ERROR, 33
  EVENT, 33
  Nodo, 33
  Server, 34

Tenedor, 34
Usuario, 34
WARN, 33
logs
Logger, 12
logv
Logger, 12

main
main.cpp, 35
main.cpp, 34
main, 35
MAXBUF, 35
MAXBUF
Client.hpp, 32
main.cpp, 35

Nodo
Logger.hpp, 33

Parser, 13
~Parser, 14
Figuras, 15
getFiguras, 14
getPiezas, 14
Parser, 14
piezas, 15
procesarFiguras, 14
procesarPiezas, 15
Parser.cpp, 36
Parser.hpp, 36
pedirFigura
Client, 10
piezas
Parser, 15
port
VSocket, 30
procesarFiguras
Parser, 14
procesarPiezas
Parser, 15

Read
Socket, 17
SSLSocket, 23
VSocket, 27
recvFrom
VSocket, 28

sendTo
VSocket, 28
Server
Logger.hpp, 34
ShowCerts
SSLSocket, 23
Socket, 15
~Socket, 17
Connect, 17
Read, 17
Socket, 16
Write, 19
Socket.cpp, 37
Socket.hpp, 37
sockId
VSocket, 30
SSLSocket, 19
~SSLSocket, 21
BIO, 25
Connect, 22
Context, 25
GetCipher, 22
InitContext, 22
InitSSL, 23
LoadCertificates, 23
Read, 23
ShowCerts, 23
SSLSocket, 21
Write, 24
SSLSocket.cpp, 38
SSLSocket.hpp, 38

Tenedor
Logger.hpp, 34
tiempo
Logger, 12
tm
Logger, 13
TryToConnect
VSocket, 28, 29
type
VSocket, 30

Usuario
Logger.hpp, 34

VSocket, 25
~VSocket, 26
Bind, 26
Close, 26
Connect, 26, 27
Init, 27
IPv6, 30
port, 30
Read, 27
recvFrom, 28
sendTo, 28
sockId, 30
TryToConnect, 28, 29
type, 30
Write, 29
VSocket.cpp, 39
VSocket.hpp, 39

WARN
Logger.hpp, 33
Write
Socket, 19
SSLSocket, 24
VSocket, 29